

Design Report

Optimistic TCP ACK Attack on a Media-Streaming Server

Al Shahriar Alif (2105158)

Imdadul Hasan (2105160)

CSE 406 — Cyber Security

1 Overview of the Project Idea

TCP does not measure the network directly. It *infers* how much data the path can hold from the acknowledgements (ACKs) the receiver sends back: every ACK that advances the acknowledged byte number tells the sender “this much data has left the network, you may send more.” This feedback loop—often called the *ACK clock*—drives both the timing of new transmissions and the growth of the congestion window (*cwnd*). The sender trusts these ACKs implicitly; nothing in the base protocol proves that the acknowledged bytes were ever received.

The **optimistic ACK attack** abuses exactly this trust. A malicious client opens a normal TCP connection to a server and then returns ACKs for data that has *not yet arrived* (or will never arrive). Each forged ACK advances the sender’s “lowest unacknowledged byte” pointer (*snd_una*), tricking the server into believing the path is emptying faster than it really is. The server responds by opening *cwnd* and pushing bytes onto the wire faster than the bottleneck can drain them.

In this project we build the attack in an isolated Mininet/Docker testbed against an NGINX server streaming a video file over HTTP, and we study three things concretely:

1. **Sender reaction.** How the Linux TCP stack’s *cwnd*, send rate, and retransmissions change when it is fed optimistic ACKs, and how TCP CUBIC (Linux default) differs from TCP Reno.
2. **Collateral damage.** How a second, *honest* client streaming from the same server—sharing the same bottleneck link—loses throughput and suffers video rebuffering while the attack runs.
3. **Defense.** A server-side detector that spots physically impossible ACKs (acknowledging bytes never sent, or advancing faster than the link allows) and drops them before they reach the TCP state machine.

What “success” means here. We are not chasing the textbook myth of an infinite congestion window. Modern Linux caps the damage with pacing, socket send-buffer limits, and loss recovery. Success for us is a *measurable* inflation of the server’s offered load and a *measurable* degradation of the honest client’s quality of experience (QoE)—both attributable to the forged ACKs and both removed by our defense.

Scope and safety. Everything runs on one workstation inside Mininet network namespaces and Docker containers. No packet leaves the host; no external, campus, or third-party system is touched.

2 Network Topology, Components and Timing Diagram

2.1 Topology and components

The testbed is a single bottleneck (“dumbbell”) topology. The server reaches the client subnet only through one constrained link, so both the attacker and the honest client compete for the same 10 Mbps of capacity—this shared bottleneck is what turns the attacker’s over-sending into damage for the honest flow.

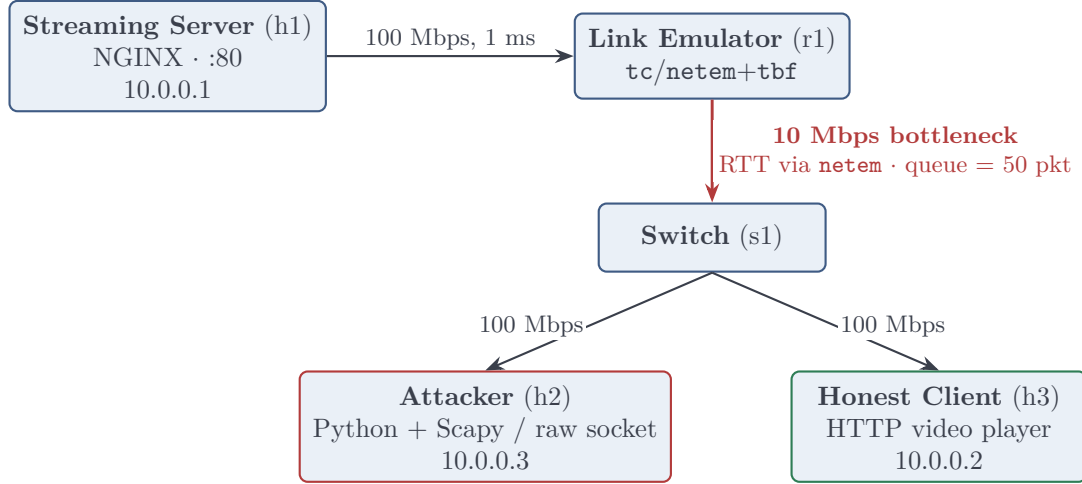


Figure 1: Single-bottleneck testbed. The red link (r1→s1) is the only 10 Mbps path to the client subnet and is shared by the attacker (h2) and the honest client (h3).

Node	Role	Address	Key software / configuration
h1	Media server (victim sender)	10.0.0.1:80	NGINX; MP4 file; <i>cwnd</i> read via <code>ss -ti</code>
h2	Attacker client	10.0.0.3	Python 3 + Scapy raw injection; <code>iptables</code> RST drop
h3	Honest client	10.0.0.2	HTTP GET loop / player; goodput + stall logger
r1	Link emulator / gateway	—	<code>tc: tbf</code> (rate) + <code>netem</code> (delay), 50-packet queue
s1	Layer-2 switch	—	Mininet OVS bridge to the client hosts

Table 1: Components and their configured roles.

2.2 Timing diagram of the attack

Figure 2 shows the message timeline. Time flows downward. The attacker first behaves like an ordinary client (handshake + request), then switches to injecting ACKs whose acknowledgement number runs *ahead* of the bytes actually delivered. The server treats those ACKs as proof of delivery and over-sends into the bottleneck.

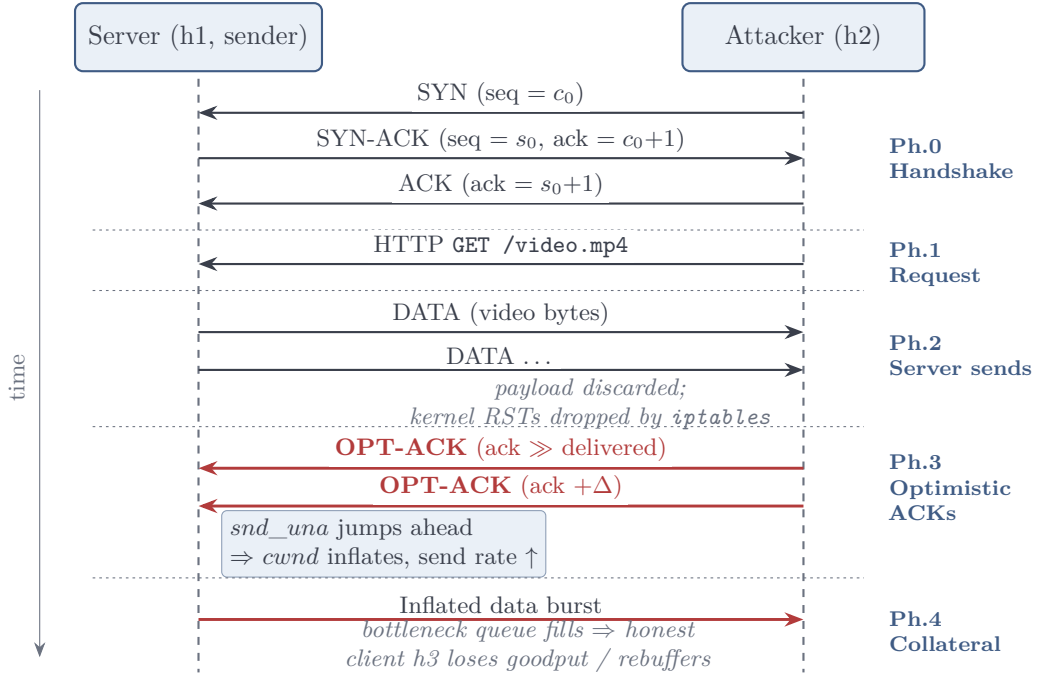


Figure 2: Attack timeline. Phases 0–1 are indistinguishable from a normal client; the forged ACKs begin in Phase 3 and cause the over-send and collateral damage in Phase 4.

3 Packet and Frame Structure

The attacker never touches the link-layer or IP framing—those carry its real MAC/IP so replies route back to it. The forgery lives entirely in the **TCP header**, specifically the *Acknowledgement Number* (and a comfortably large *Window*). This section shows how a forged ACK sits inside the stack.

3.1 Encapsulation

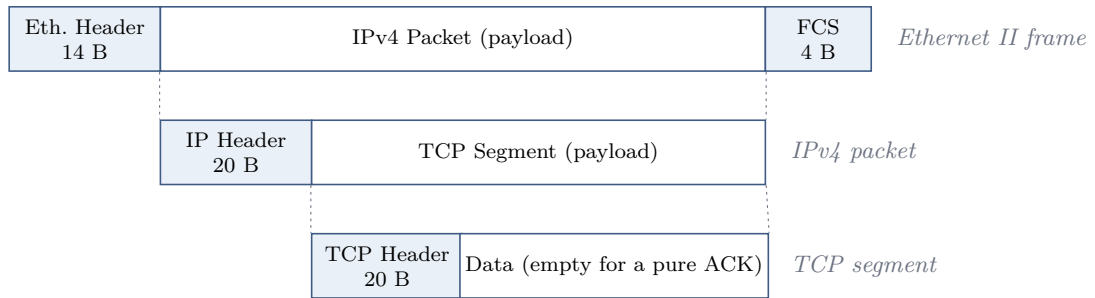


Figure 3: Encapsulation of a forged optimistic ACK. It is a *pure* ACK: valid Ethernet/IP framing, a TCP header with the ACK flag set, and no application payload.

3.2 IPv4 header (unchanged by the attack)

0	8	16	24	31
Ver	IHL	DSCP/ECN	Total Length	
Identification			Flg	Fragment Offset
TTL		Protocol=6	Header Checksum	
Source Address = 10.0.0.3 (attacker)				
Destination Address = 10.0.0.1 (server)				
Options + Padding (usually absent)				

Figure 4: IPv4 header. The attacker uses its own real source address; nothing here is spoofed.

3.3 TCP header (the forged fields)

0	8	16	24	31
Source Port (ephemeral)		Dest Port = 80		
Sequence Number = our own <i>snd_next</i> (truthful)				
Acknowledgement Number = $s_0 + 1 + k\Delta$ (FORGED , runs ahead)				
Data Ofs	Reserved	Flags: ACK	Window (large)	
Checksum (recomputed, valid)		Urgent Pointer = 0		
Options (e.g. Timestamp echo, kept plausible)				

Figure 5: TCP header of the crafted ACK. Red fields are attacker-controlled: the *Acknowledgement Number* advances past what was truly received, and a large *Window* ensures the server is never receiver-limited.

3.4 Field values of the crafted optimistic ACK

Let s_0 be the server’s initial sequence number (learned from the SYN-ACK), c_0 our own initial sequence number, and Δ the byte step we advance per forged ACK. Table 2 lists a concrete pure-ACK packet.

Field	Value	Reason
IP src / dst	10.0.0.3 / 10.0.0.1	Real addresses; replies must return to us.
IP protocol	6 (TCP)	Ordinary TCP segment.
TCP src / dst port	ephemeral / 80	Same 4-tuple as the established connection.
TCP flags	ACK only	Pure ACK: no SYN/FIN/RST, no payload.
Sequence number	$c_0 + 1 + \text{GET} $	Truthful—we really did send only the request.
Acknowledgement no.	$s_0 + 1 + k\Delta$	Forged. Advances beyond bytes actually received.
Window	65 535 (scaled)	Keep the server from ever stalling on receive window.
Checksum	recomputed	Must be valid or the segment is dropped.

Table 2: The k -th optimistic ACK. Only the acknowledgement number (and an inflated window) deviate from an honest client’s ACK.

4 Implementation Plan

4.1 Testbed and victim service

We script the topology of Figure 1 with Mininet’s Python API and shape the bottleneck with `tc`:

- **Bottleneck:** `tbif` caps `r1→s1` at 10 Mbps; `netem` adds one-way delay for the target RTT (20/50/100 ms) and a 50-packet tail-drop queue.
- **Congestion control:** selected per run with `sysctl -w net.ipv4.tcp_congestion_control=cubic` (or `reno`). CUBIC is the default; Reno is the comparison baseline.
- **Victim:** NGINX serves a multi-hundred-MB `/video.mp4`. The honest client (h3) fetches it in a loop and logs received bytes per second; a real player can be swapped in for QoE.

4.2 Attacker tool architecture

The attacker is a user-space program that owns one TCP connection end-to-end. Because a raw-socket ACK does not correspond to a kernel-managed socket, the kernel would try to RST the “unknown” connection; we suppress that with a Netfilter rule (Sec. 4.4).

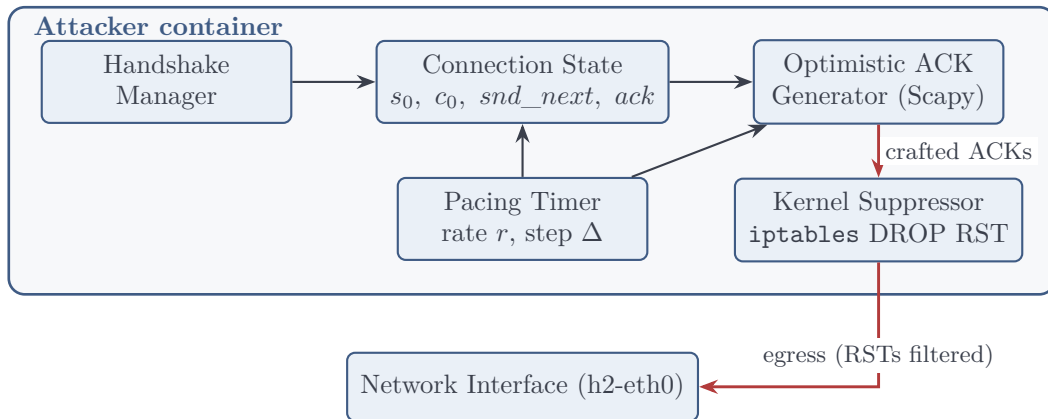


Figure 6: Attacker tool. The Handshake Manager brings up the connection and records s_0 ; the Generator then emits paced ACKs whose acknowledgement number climbs by Δ each tick while the Suppressor keeps the kernel from resetting the flow.

4.3 Attack algorithm

Algorithm 1 — Optimistic ACK injection

```
Input : server (ip,port); step DELTA (bytes); rate r (ACKs/s); window W
1 (s0, c0, sport) <- complete 3-way handshake to server      # learn server ISN s0
2 send HTTP "GET /video.mp4"                                # small request
3 snd_next <- c0 + 1 + len(GET)
4 ack <- s0 + 1
5 repeat                                                    # main injection loop
6     craft segment: flags=ACK, seq=snd_next, ack=ack, win=W
7     send via raw socket (kernel RST for this flow is DROPped)
8     ack <- ack + DELTA      # acknowledge bytes not (yet) delivered
9     sleep(1 / r)
10 until stop                                              # DELTA*r chosen > 10
    Mbps
```

We choose Δ and r so the *claimed* delivery rate $\Delta \cdot r$ comfortably exceeds the 10 Mbps bottleneck; that is what forces the server to out-run the link. A concrete Scapy core for lines 6–9:

```
from scapy.all import IP, TCP, send
ack = s0 + 1
while running:
    seg = IP(src="10.0.0.3", dst="10.0.0.1") / \
        TCP(sport=sport, dport=80, flags="A",
            seq=snd_next, ack=ack, window=65535)
    send(seg, verbose=0)      # raw injection; matching kernel RST is dropped
    ack += DELTA              # advance past what was actually received
    time.sleep(1.0 / RATE)
```

4.4 Host-kernel control

The connection is driven from user space, so the kernel must not interfere. A single outbound rule drops the spurious resets the stack generates for the raw-socket flow:

```
iptables -A OUTPUT -p tcp --tcp-flags RST RST -s 10.0.0.3 -j DROP
```

Running the attacker inside its own Docker network namespace keeps this rule (and the raw traffic) off the rest of the host. Raw injection uses an `AF_PACKET/SOCK_RAW` socket (Scapy's `send` at layer 3).

4.5 Measurement harness

Each run is repeated 5 times; we report means with error bars. Collection is lightweight so it does not perturb the flow:

What we measure	Where	Tool / cadence
<i>cwnd</i> , <i>ssthresh</i> , RTT, retransmits	Server (h1)	<code>ss -ti</code> sampled every 100 ms (eBPF if finer needed)
Bottleneck queue depth, drops	Emulator (r1)	<code>tc -s qdisc</code> every 100 ms
Offered / send rate	Server (h1)	bytes-sent delta from socket stats
Honest-client goodput, stalls	Client (h3)	app logger every 1 s
Full packet trace	r1 / h1	<code>tcpdump</code> → Wireshark/ <code>tshark</code> offline

Table 3: Telemetry sources spanning sender, network, and receiver.

4.6 Experiment procedure

We compare three conditions with everything else fixed:

1. **Baseline** — honest client only (plus an honest background flow). Establishes the fair-share reference.
2. **Attack** — baseline plus the attacker, sweeping TCP variant (Reno/CUBIC), RTT (20/50/100 ms), and $\Delta \cdot r$.
3. **Defended** — attack condition with the Section 6 detector (and, separately, NGINX `limit_rate`) enabled.

5 Expected Outcome of a Successful Attack

5.1 On the server (sender)

Once optimistic ACKs begin, the server sees its data “leave the network” almost instantly, so it leaves slow start early and pushes *cwnd* far past the path’s bandwidth–delay product (BDP ≈ 42 MSS at 10 Mbps / 50 ms). Growth is not unbounded—the socket send buffer, pacing, and eventually real losses cap it—but the plateau sits well above the honest ceiling.

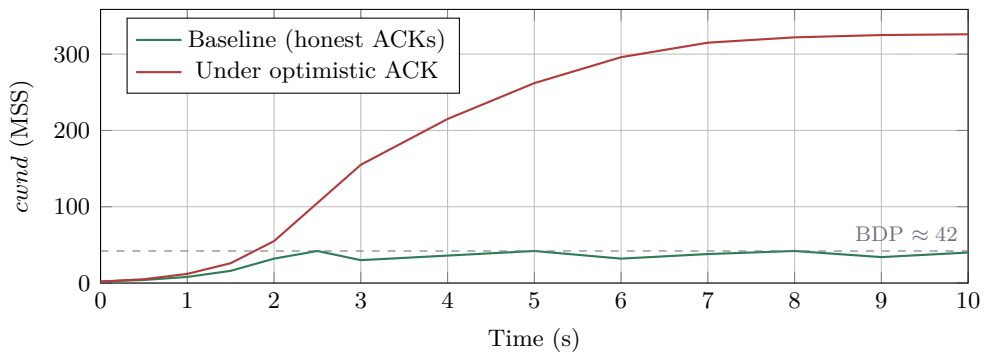


Figure 7: Expected (schematic) *cwnd* trajectory. Honest traffic oscillates around the BDP; forged ACKs drive *cwnd* several times higher until buffer/pacing limits cap it. Illustrative, not measured.

5.2 On the honest client (collateral damage)

The extra bytes have to go somewhere: they pile into r1’s 50-packet queue. Once the queue saturates, the honest flow’s packets are delayed and dropped, its own *cwnd* collapses, and the

video stalls. This is the result that matters for the report—the attacker degrades a bystander.

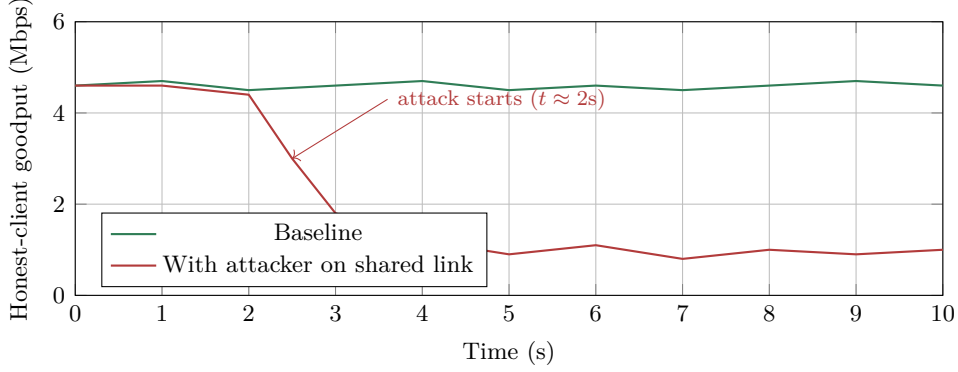


Figure 8: Expected (schematic) collapse of the honest client’s goodput once the attacker saturates the shared queue. Illustrative, not measured.

5.3 Cause-and-effect summary

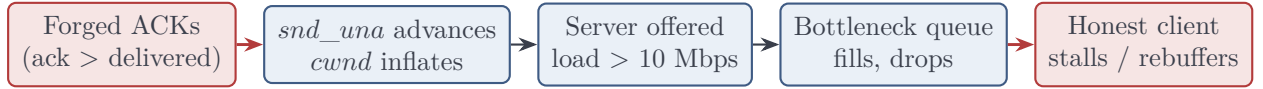


Figure 9: The chain from a single forged field to real user-visible harm.

Concretely we expect: (i) CUBIC to inflate faster and higher than Reno because its window function is time-driven, not purely ACK-counted; (ii) larger $\Delta \cdot r$ to raise queue occupancy and honest-client loss; and (iii) a rise in server-side retransmissions late in the attack, when bytes the server “believes” were delivered are in fact lost and must be resent.

6 Defense Idea

6.1 Core idea: ACK plausibility

An honest cumulative ACK obeys two physical facts that the attacker cannot satisfy at once:

- (1) Sent-bound: $ack_no \leq snd_max$,
- (2) Rate-bound: $\frac{ack_no(t_2) - ack_no(t_1)}{t_2 - t_1} \leq B_{\text{path}}$,

where snd_max is the highest byte the server has actually transmitted and B_{path} is the physical path bandwidth (10 Mbps here, converted to bytes/s). Rule (1) catches ACKs for bytes never sent; rule (2) catches ACKs that advance faster than the link could ever deliver—the signature of optimistic ACKing. A short leading margin/hysteresis on (2) avoids flagging normal bursts.

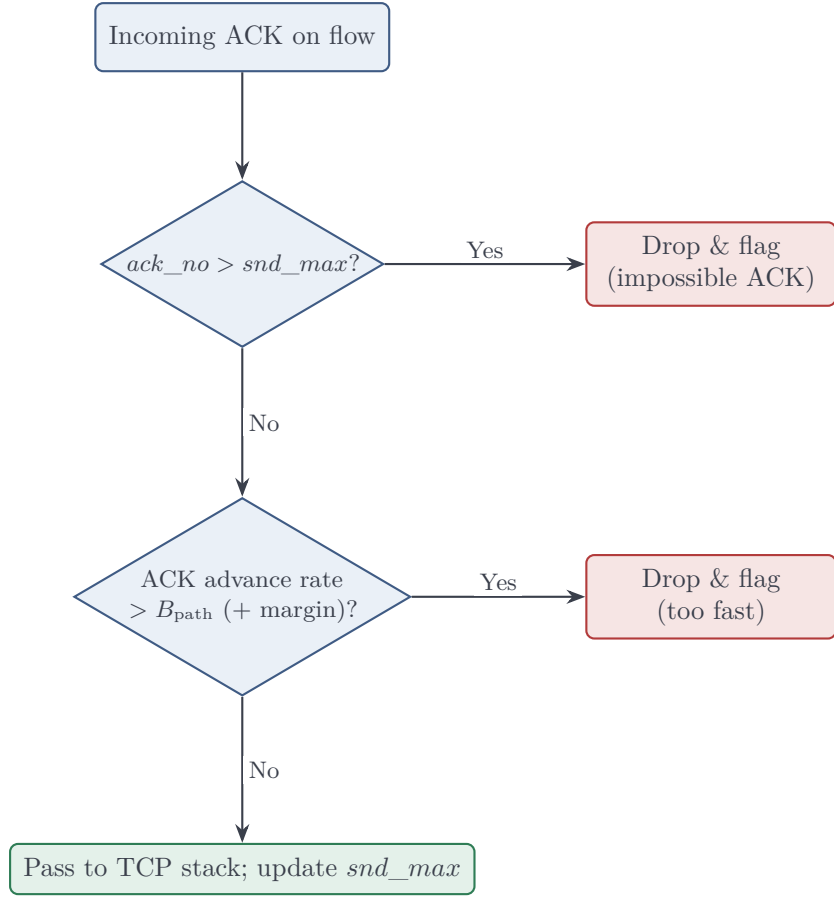


Figure 10: Detector decision flow. Both checks are per-connection and run before the ACK reaches the TCP state machine.

6.2 Where it runs

The detector must see and drop an ACK *before* the kernel processes it—a passive `ss` monitor is too late. We place it on the server’s ingress path as a Netfilter hook (or eBPF/TC/XDP), keeping a small per-flow table of snd_max and the last (ack_no, t) sample.

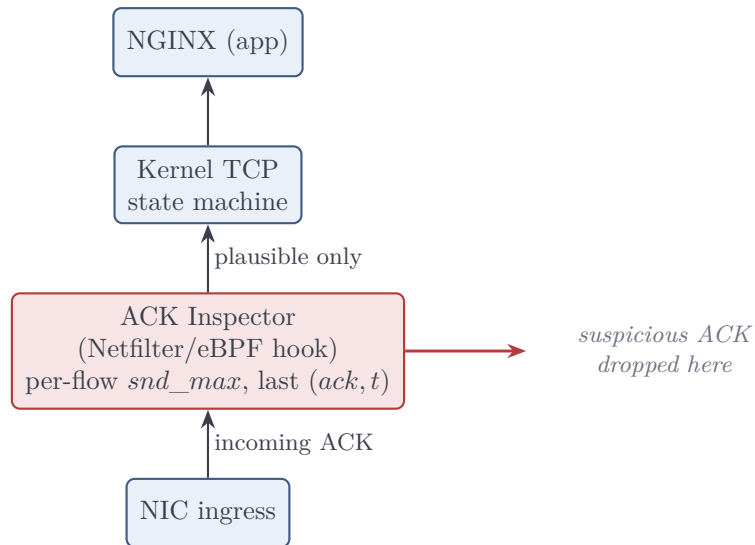


Figure 11: Detector placement: it intercepts ACKs on ingress and forwards only plausible ones to the TCP stack, so forged ACKs never move *snd_una*.

6.3 Application-layer backstop

As a complementary, coarser control we cap the media location in NGINX:

```
location /media/ { limit_rate 2m; } # 2 MB/s ceiling per connection
```

This limits how fast NGINX hands bytes to the socket, so it *bounds* the damage an over-eager sender can do. It is an impact limiter, not a fix: it does not stop the forged ACKs from moving TCP’s state, which is why the plausibility detector is our primary defense.